

MENDSPEECH REVISED LEARNING SYSTEM

Week 5: Streaming, Cache Aware Inference, and Adaptive Context

Turn the recognizer into a real time system and test uncertainty guided context spending.

Six focused build days plus one consolidation day. Each day ends with named artifacts or measured evidence.

Week milestone

Turn the recognizer into a real time system and test uncertainty guided context spending.

Week map

Day	Focus	Minimum evidence	Compute
Day 29	Offline versus streaming ASR	• Compare offline transcript with naive chunk by chunk transcription.	Modal L4
Day 30	Buffered streaming	• Sweep buffer and stride settings. • Measure WER and latency tradeoffs.	Modal L4
Day 31	Cache aware streaming internals	• Compare buffered and cache aware inference on the same audio and same hardware.	Modal L4
Day 32	Lookahead ablation	• Plot WER versus latency and identify dominated operating points.	Modal L4
Day 33	Break the cache on purpose	• Measure WER changes around the reset point. • Inspect whether errors cluster near boundaries or propagate later.	Modal L4
Day 34	Adaptive context controller prototype	• Compare fixed fast, fixed accurate, and adaptive policies on a controlled subset.	Modal L4
Day 35	Week 5 live streaming milestone	• Record a short demo with clean and damaged speech. • Document remaining technical limitations honestly.	Modal L4

Reference spine

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Offline versus streaming ASR

LEARN

- Audio chunks.
- Algorithmic latency.
- Partial hypotheses.
- Endpointing and finalization.

BUILD IN MENDESPREECH

- Create a chunk simulator that feeds audio incrementally.
- Log when each chunk becomes available and when text changes.

EXPERIMENT AND MEASURE

- Compare offline transcript with naive chunk by chunk transcription.

REQUIRED OUTPUT

- src/streaming/chunker.py
- results/day29_offline_vs_naive.csv

Completion check: You can explain why naive chunking creates boundary errors and redundant compute.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Buffered streaming

LEARN

- Overlapping windows.
- Buffer size.
- Stride.
- Repeated computation.

BUILD IN MENDSPEECH

- Implement or run buffered streaming with configurable overlap.
- Measure how much audio is recomputed.

EXPERIMENT AND MEASURE

- Sweep buffer and stride settings.
- Measure WER and latency tradeoffs.

REQUIRED OUTPUT

- `src/streaming/buffered.py`
- `results/day30_buffered_sweep.csv`

Completion check: You can quantify the compute waste caused by overlapping history.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Cache aware streaming internals

LEARN

- Cached activations.
- Past context state.
- Streaming masks.
- Right context and lookahead.

BUILD IN MENDSPEECH

- Use NeMo cache aware streaming inference on a supported FastConformer checkpoint.
- Log cache related configuration and chunk boundaries.

EXPERIMENT AND MEASURE

- Compare buffered and cache aware inference on the same audio and same hardware.

REQUIRED OUTPUT

- src/streaming/cache_aware_runner.py
- results/day31_buffered_vs_cache.csv

Completion check: You can explain what is cached, what is recomputed, and why cache aware inference can be more efficient.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Lookahead ablation

LEARN

- Right context.
- Lookahead.
- Commit delay.
- WER and latency as competing objectives.

BUILD IN MENDESPREECH

- Run several supported lookahead settings with everything else fixed.
- Store per utterance and aggregate metrics.

EXPERIMENT AND MEASURE

- Plot WER versus latency and identify dominated operating points.

REQUIRED OUTPUT

- experiments/lookahead_ablation.py
- results/day32_lookahead.csv
- results/day32_pareto.png

Completion check: You can defend a Balanced operating point using data rather than preference.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Break the cache on purpose

LEARN

- State continuity.
- Chunk boundary dependencies.
- Cache reset and truncation.

BUILD IN MENDSPEECH

- Add controlled experiments that reset or shorten cache at selected boundaries.

EXPERIMENT AND MEASURE

- Measure WER changes around the reset point.
- Inspect whether errors cluster near boundaries or propagate later.

REQUIRED OUTPUT

- experiments/cache_break_test.py
- results/day33_cache_failures.md

Completion check: You can explain a concrete failure caused by incorrect state handling.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Adaptive context controller prototype

LEARN

- Policy driven context selection.
- Confidence smoothing.
- Latency budget.
- Stability versus oscillation.

BUILD IN MENDESPREECH

- Implement a controller that classifies chunks as easy or uncertain.
- Map states to small or larger supported right context settings, even if the first prototype must simulate switching between runs.

EXPERIMENT AND MEASURE

- Compare fixed fast, fixed accurate, and adaptive policies on a controlled subset.

REQUIRED OUTPUT

- src/controller/adaptive_context.py
- results/day34_adaptive_context.csv

Completion check: You have a falsifiable first answer to whether uncertainty can guide context spending.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples

Week 5 live streaming milestone

LEARN

- Review buffered streaming, cache aware inference, lookahead, cache failures, and adaptive context.

BUILD IN MENDSPEECH

- Connect microphone or simulated live audio to the streaming recognizer.
- Show partial text, confidence timeline, current context mode, and latency.

EXPERIMENT AND MEASURE

- Record a short demo with clean and damaged speech.
- Document remaining technical limitations honestly.

REQUIRED OUTPUT

- app/mendspeech_v2_streaming.py
- demos/week5_streaming_demo.mp4
- reports/week5_streaming.md

Completion check: A person can speak and watch MendSpeech transcribe incrementally while exposing the state that drives repair decisions.

Study method

25 minutes focused reading. 65 minutes implementation or controlled experiment. 20 minutes research notebook. 10 minutes commit and explain the result aloud. When debugging is incomplete, continue the same task in the next session instead of pretending the day is finished.

REFERENCE PRIORITY

- Stateful or cache aware Conformer primary material
- NVIDIA NeMo streaming ASR documentation and examples